

Aqui está um **resumo completo, detalhado e bem estruturado** da **implementação híbrida da integração Catapult OpenField**, escrito de forma clara para ser usado como *contexto de um IA Agent*.

Inclui **contexto, atores, fluxo, requisitos e responsabilidades técnicas**.

Resumo Detalhado — Integração Híbrida Catapult OpenField (para IA Agent)

1. Contexto Geral

A aplicação tem como objetivo **receber automaticamente dados de desempenho esportivo** provenientes dos dispositivos GPS Catapult (OpenField), usados por jogadores de futebol durante treinos e jogos.

Cada **clube cliente** da aplicação utiliza seu próprio ambiente OpenField e gera seus próprios tokens de API e Webhook Secrets.

A integração deve:

- permitir que **cada clube conecte sua própria conta Catapult**, sem compartilhamento de credenciais entre clubes;
 - receber dados via **webhooks enviados pela Catapult**;
 - validar os dados recebidos;
 - processar e converter métricas (ex.: m/s → km/h);
 - armazenar no backend (Supabase) de forma organizada e por clube;
 - garantir segurança e isolamento total entre clubes.
-

2. Modelo de Integração: Híbrido

O modelo híbrido combina:

A) Cadastro manual controlado pelo clube

- Cada clube solicita à Catapult:
 - **API Token** (Access Token)
 - **Webhook Secret**
 - Permissão para usar Webhooks
- O clube informa esses valores para o sistema.

B) Configuração centralizada do Webhook

O sistema gera:

- um **ClubID** interno
- uma URL de webhook única para cada clube:

`https://suaapi.com/webhooks/catapult/{clubId}`

O clube configura essa URL no dashboard do OpenField.

3. Fluxo Completo de Conexão de um Clube

1. Clube acessa sua área administrativa

O usuário (comissão técnica) acessa a interface da sua plataforma.

2. Ele clica em “Conectar Catapult OpenField”

3. O sistema exibe instruções para o processo manual

- solicitar o acesso à API OpenField Support
- habilitar Webhooks

- gerar API Token
- gerar Webhook Secret

4. Clube preenche o formulário de credenciais

Campos:

- API Token
- Webhook Secret
- Organization ID (opcional)
- Nome do clube

5. Backend valida token com a API Catapult

- Faz chamada `/me` ou equivalente para validar credencial
- Se OK → credencial é salva criptografada no Supabase

6. Backend gera a URL de Webhook

Ex:

POST `https://api.meusistema.com/webhooks/catapult/97ba2e85-1aab-45e1`

Aparece na interface para que o clube copie e cole no painel Catapult.

7. Clube cadastra o webhook na OpenField

- usando a URL fornecida
- usando o Secret fornecido por ele mesmo

8. Sempre que um atleta subir dados de treino

O OpenField envia:

- evento de “session completed”, “activity uploaded”, etc.
- payload com dados brutos
- assinatura HMAC usando o secret

9. Backend recebe webhook

O fluxo backend é:

1. Identifica o clube pelo endpoint (`/webhooks/catapult/{clubId}`)
2. Valida assinatura HMAC
3. Faz parse do payload
4. Envia para módulo de processamento

10. Processamento dos dados

Converte:

- velocidade → km/h
- distância → metros ou km
- aceleração/deceleração
- carga TRIMP
- PlayerLoad
- etc.

Normaliza para um formato padrão interno.

11. Armazena dados no Supabase

Tabelas:

- athletes

- sessions
- metrics
- raw_payloads (backup opcional)

12. Dados ficam disponíveis para dashboards, APIs internas, relatórios, IA etc.

4. Responsabilidades Técnicas

Backend (Deno + TypeScript + Supabase)

- Criar endpoint para registrar credenciais de clubes
- Validar tokens com OpenField
- Criar e prover URL de webhook por clube
- Processar e validar Webhooks
- Normalização dos dados
- Persistência estruturada no Supabase
- Logs e monitoramento
- Modos de retry para webhooks

Segurança

- Tokens armazenados criptografados (AES-256 GCM)
- Webhook Secret nunca exibido de novo após cadastro
- Validação obrigatória HMAC SHA-256
- Rate limiting por clube

- ClubID como UUID não previsível
-

5. Tipos de Dados Recebidos da Catapult (Esperado)

Dependendo do nível de acesso, a Catapult envia:

Dados de sessão

- identificação da sessão
- horário, duração
- fase do treino

Dados de atletas

- ID do atleta
- posição, nome
- dispositivo usado

Dados de performance

- velocidades instantâneas
- distância total
- distância em zonas (Z1–Z6)
- acelerações / desacelerações
- PlayerLoad
- impactos G
- “High Speed Running”

- intensidade GPS
- curvas, sprint count
- etc.

Esses valores geralmente vêm em:

- m/s
- metros
- raw counts

O backend converte para:

- km/h
- km
- métricas normalizadas

6. Vantagens do Modelo Híbrido

✓ Independência por clube

Cada clube controla seu próprio token e secret.

✓ Segurança total

Seu sistema nunca acessa as credenciais administrativas do clube.

✓ Escalável

Adicionar novos clubes é simples e padronizado.

✓ Baixa responsabilidade legal

Você não manipula a autenticação interna da Catapult – os clubes são responsáveis.

✓ Menor suporte

Problemas com o OpenField são responsabilidade do clube, não suas.

7. Riscos e Considerações

- Dependência de procedimentos manuais por parte do clube.
 - Alguns clubes podem não saber configurar o webhook → criar tutorial.
 - Necessário bom tratamento de erros quando tokens expirarem (geralmente não expiram).
 - Testes devem ser feitos usando **Catapult Sandbox ou conta trial**.
-

8. Conclusão

A integração híbrida Catapult OpenField é segura, escalável e simples de operar.

Cada clube habilita sua própria API e Webhook Secret, enquanto sua aplicação fornece uma URL única e organiza todo o processamento e armazenamento.

O backend recebe e trata automaticamente todos os dados de treino, convertendo as métricas e armazenando-as prontas para uso.

Se quiser, posso gerar também:

- ✓ arquitetura visual da integração
- ✓ estrutura de tabelas no Supabase
- ✓ endpoints REST completos
- ✓ checklist de segurança
- ✓ esquema do fluxo de webhooks
- ✓ prompt ideal para o seu AI Agent

Só dizer “quero”.

A sua principal preocupação — não ter uma conta de desenvolvedor ou `API Token` da OpenField agora — não é um impedimento.¹ Como você corretamente apontou, podemos construir e testar ~70% de toda a infraestrutura com dados *mockados* (simulados).

A única parte que não podemos *verificar* até termos um token real é se o `API Token` padrão do clube tem permissão para *criar webhooks programaticamente* (o `POST /openfield/v1/webhooks`).²

Abaixo está o roteiro de implementação detalhado, passo a passo, focado em construir toda a base técnica *agora*, sem depender de um token.

Roteiro de Implementação: Base Técnica (Sem Token)

Vamos seguir a estrutura que você definiu.

①, ②, ③: Estrutura do Banco Supabase (SQL)

Esta é a fundação. Podemos construir 100% disso agora. Vamos ao seu editor SQL no painel do Supabase.

O esquema que você sugeriu é ótimo. Vamos refiná-lo para usar a prática recomendada do **Supabase Vault** para armazenar os tokens.⁴ Em vez de criptografar o token diretamente na tabela `clubs`, armazenaremos um *ponteiro* para um segredo no Vault. É mais seguro e mais fácil de gerenciar.⁶

1. Habilitar Extensões (SQL):

Execute isso primeiro (se já não estiverem ativas).

SQL

```
-- Habilita o Vault para armazenamento seguro de chaves
```

```
create extension if not exists pgsodium;
```

```
create extension if not exists vault;
```

```
-- Habilita o pg_net para Edge Functions chamarem a si mesmas (ou outras)
```

```
create extension if not exists pg_net;
```

```
-- Habilita o TimescaleDB para a tabela 'metrics'
```

```
-- (Verifique se está disponível no seu plano)
```

```
create extension if not exists timescaledb;
```

2. Criar as Tabelas (SQL):

SQL

```
-- Tabela de Clubes (Tenants)
CREATE TABLE public.clubs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Tabela de Integrações (Armazena as chaves e IDs)
CREATE TABLE public.club_integrations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  club_id UUID NOT NULL REFERENCES public.clubs(id) ON DELETE CASCADE,
  provider TEXT NOT NULL DEFAULT 'catapult',

  -- ID público e não sequencial para a URL do webhook
  webhook_uuid UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

  -- ID do webhook retornado pela Catapult (para referência)
  catapult_webhook_id TEXT,

  -- PONTEIROS SEGUROS para os segredos no Vault
  api_key_vault_id UUID REFERENCES vault.secrets(id),
  webhook_secret_vault_id UUID REFERENCES vault.secrets(id),

  UNIQUE(club_id, provider)
);

-- Tabela de Atletas (Mapeamento De-Para)
CREATE TABLE public.athletes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  club_id UUID NOT NULL REFERENCES public.clubs(id),
  full_name TEXT,

  -- O ID do atleta NO SISTEMA DA CATAPULT (ex: 'playerId' tag)
  external_catapult_id TEXT,

  UNIQUE(club_id, external_catapult_id)
);

-- Tabela de Sessões de Treino
CREATE TABLE public.sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  club_id UUID NOT NULL REFERENCES public.clubs(id),
```

```

athlete_id UUID NOT NULL REFERENCES public.athletes(id),

-- O ID da sessão NO SISTEMA DA CATAPULT
external_session_id TEXT NOT NULL,

started_at TIMESTAMPTZ,
ended_at TIMESTAMPTZ,
raw_json JSONB, -- Ótimo para 'provider_metadata' [9]

UNIQUE(club_id, external_session_id, athlete_id)
);

-- Tabela de Métricas (Time-Series)
CREATE TABLE public.metrics (
  session_id UUID NOT NULL REFERENCES public.sessions(id) ON DELETE CASCADE,
  athlete_id UUID NOT NULL REFERENCES public.athletes(id),
  "timestamp" TIMESTAMPTZ NOT NULL,
  metric TEXT NOT NULL,
  value NUMERIC NOT NULL,
  unit TEXT
);

-- Converter 'metrics' em uma Hypertable do TimescaleDB
SELECT create_hypertable('metrics', 'timestamp');

-- Tabela de Logs (como você sugeriu)
CREATE TABLE public.webhook_logs (
  id BIGSERIAL PRIMARY KEY,
  received_at TIMESTAMPTZ DEFAULT now(),
  integration_id UUID REFERENCES public.club_integrations(id),
  club_id UUID REFERENCES public.clubs(id),
  signature_valid BOOLEAN,
  status_code INT,
  request_body TEXT
);

```

3. Criar Funções RPC para o Vault (SQL):

Edge Functions não podem acessar o Vault diretamente.¹⁰ Elas devem chamar funções de banco de dados SECURITY DEFINER.⁴

SQL

```

-- Função para CRIAR um segredo (retorna o ID do segredo)
CREATE OR REPLACE FUNCTION create_team_secret(secret_value TEXT, secret_name TEXT)
RETURNS uuid

```

```

LANGUAGE plpgsql
SECURITY DEFINER -- Executa com privilégios de 'postgres'
SET search_path = vault, public
AS $$
BEGIN
    RETURN vault.create_secret(secret_value, secret_name);
END;
$$;

-- Função para LER um segredo (retorna o valor decifrado)
CREATE OR REPLACE FUNCTION get_secret_by_id(secret_id_in UUID)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = vault, public
AS $$
DECLARE
    decrypted_secret_value TEXT;
BEGIN
    SELECT decrypted_secret
    INTO decrypted_secret_value
    FROM vault.decrypted_secrets
    WHERE id = secret_id_in;
    RETURN decrypted_secret_value;
END;
$$;

```

Base de dados concluída. 100% feito, 0% de dependência do token.



🔗: Interface "Conectar Catapult" no Flutter

Podemos construir a tela inteira e a lógica de chamada de API.

Arquivo: `lib/screens/connect_catapult_screen.dart` (Exemplo)

Dart

```

import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class ConnectCatapultScreen extends StatefulWidget {
    const ConnectCatapultScreen({Key? key}) : super(key: key);

    @override
    _ConnectCatapultScreenState createState() => _ConnectCatapultScreenState();

```

```
}
```

```
class _ConnectCatapultScreenState extends State<ConnectCatapultScreen> {  
  final _tokenController = TextEditingController();  
  bool _isLoading = false;  
  String? _statusMessage;  
  
  Future<void> _connectClub() async {  
    setState(() {  
      _isLoading = true;  
      _statusMessage = 'Conectando...';  
    });  
  
    final supabase = Supabase.instance.client;  
  
    try {  
      // 1. Chama a Edge Function que vamos criar (Passo 4)  
      // O token de autenticação do admin do time é enviado automaticamente [13]  
      final response = await supabase.functions.invoke(  
        'catapult-connect', // Nome da nossa função de backend  
        body: {'api_token': _tokenController.text.trim()},  
      );  
  
      if (response.status == 200) {  
        // 2. A função (mesmo mockada) retorna sucesso  
        setState(() {  
          _statusMessage = 'Conectado com Sucesso!';  
          _isLoading = false;  
        });  
        // TODO: Navegar para a tela de "Mapeamento de Atletas" (De-Para)  
      } else {  
        throw Exception(response.data['error']?? 'Falha na conexão');  
      }  
    } catch (e) {  
      setState(() {  
        _statusMessage = 'Erro: ${e.toString()}';  
        _isLoading = false;  
      });  
    }  
  }  
}
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text("Conectar Catapult")),  
    body: Padding(  

```

```

padding: const EdgeInsets.all(16.0),
child: Column(
  children:
),
),
);
}
}

```

Frontend concluído. 100% feito, 0% de dependência do token.

④: Service de Normalização de Métricas

Podemos criar esta lógica em TypeScript/Deno agora mesmo. A sua função de webhook (Passo 5) irá importar este módulo.

Arquivo: `supabase/functions/_shared/normalization_service.ts`

TypeScript

```

// (Arquivo mock de dados para teste)
export const MOCK_CATAPULT_PAYLOAD = {
  sessionId: 'mock-session-123',
  athleteId: 'mock-player-abc', // Este é o external_catapult_id
  metrics:
};

// Interface para nossos dados normalizados
export interface NormalizedMetric {
  metric: string;
  value: number;
  unit: string;
  timestamp: string;
  zone?: string;
}

/**
 * Converte m/s para km/h
 */
function toKmH(mps: number): number {
  return mps * 3.6; //
}

/**
 * Converte metros para km

```

```

*/
function toKm(meters: number): number {
  return meters / 1000;
}

/**
 * Define a zona de velocidade (exemplo)
 */
function getVelocityZone(kmh: number): string {
  if (kmh > 24) return 'zona 5 (sprint)';
  if (kmh > 18) return 'zona 4';
  if (kmh > 12) return 'zona 3';
  if (kmh > 6) return 'zona 2';
  return 'zona 1';
}

/**
 * Processa e normaliza uma lista de métricas brutas
 */
export function normalizeMetrics(rawMetrics: any): NormalizedMetric {
  const normalized: NormalizedMetric = [];

  for (const m of rawMetrics) {
    let { metric, value, unit, timestamp } = m;
    let zone: string | undefined = undefined;

    if (metric === 'speed' && unit === 'm/s') {
      const kmh = toKmH(value);
      zone = getVelocityZone(kmh);
      // Opcional: converter a métrica principal para km/h
      // value = kmh;
      // unit = 'km/h';
    }

    if (metric === 'distance' && unit === 'm') {
      // Opcional: converter para km
      // value = toKm(value);
      // unit = 'km';
    }

    if (metric === 'player_load') {
      // PlayerLoad é uma unidade arbitrária, manter como está
    }

    normalized.push({ metric, value, unit, timestamp, zone });
  }
}

```

```
    return normalized;
}
```

Lógica de normalização concluída. 100% feito, 0% de dependência do token.

❶ & ❸: Endpoint de Conexão (/api/catapult/connect)

Agora, a lógica de *backend* que a tela do Flutter (Passo 2) irá chamar.

Arquivo: `supabase/functions/catapult-connect/index.ts`

TypeScript

```
// supabase/functions/catapult-connect/index.ts
import { createClient } from 'npm:@supabase/supabase-js@2'

// Variável de ambiente para controlar o mock
const MOCK_MODE = Deno.env.get('MOCK_MODE') === 'true';

Deno.serve(async (req) => {
  // Esta função DEVE ser autenticada
  const supabaseAdmin = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
  );

  // 1. Obter o club_id do usuário autenticado (Admin do Time)
  // (Esta é uma lógica de exemplo, você precisa implementar a obtenção do club_id do JWT [14] ou
  // tabela 'profiles')
  const { data: { user } } = await supabaseAdmin.auth.getUser(
    req.headers.get('Authorization')!.replace('Bearer ', '')
  );
  if (!user) return new Response(JSON.stringify({ error: 'Não autorizado' }), { status: 401 });

  // Ex: const { data: profile } = await supabaseAdmin.from('profiles').select('club_id').eq('id',
  // user.id).single();
  const club_id = 'uuid-do-clube-aqui'; // Substituir pela lógica real

  // 2. Obter o API Token do corpo da requisição
  const { api_token } = await req.json();
  if (!api_token) return new Response(JSON.stringify({ error: 'API Token ausente' }), { status: 400 });

  // 3. (MOCK) Validar o Token
  // No fluxo real, chamamos `GET /openfield/v1/athletes` [3, 15, 16, 1, 17, 18, 19, 20]
```



```

if (!MOCK_MODE) {
  const validationResponse = await fetch('https://connect.catapultsports.com/api/v1/athletes', { //
[16]
    headers: { 'Authorization': `Bearer ${api_token}` }
  });
  if (!validationResponse.ok) {
    return new Response(JSON.stringify({ error: 'Token de API inválido' }), { status: 401 });
  }
}

// 4. Armazenar o API Token no Vault
const { data: apiKeyVaultId, error: vaultErr1 } = await supabaseAdmin.rpc('create_team_secret', {
  secret_value: api_token,
  secret_name: `catapult_api_key_club_${club_id}`
});
if (vaultErr1) throw vaultErr1;

// 5. Gerar URL e criar registro de integração
const { data: integration, error: intErr } = await supabaseAdmin
  .from('club_integrations')
  .insert({
    club_id: club_id,
    api_key_vault_id: apiKeyVaultId,
  })
  .select('webhook_uuid')
  .single();

if (intErr) throw intErr;

const callbackUrl =
`${Deno.env.get('SUPABASE_URL')}/functions/v1/catapult-webhook/${integration.webhook_uuid}`;

// 6. (MOCK) Criar o Webhook na Catapult
// Esta é a etapa que SÓ PODE SER TESTADA com um token real.
let webhook_id: string;
let webhook_secret: string;

if (MOCK_MODE) {
  console.log(`[MOCK] Criando webhook para ${callbackUrl}`);
  webhook_id = 'mock-webhook-id-123';
  webhook_secret = 'mock-secret-key-abc-xyz'; // Segredo mockado
} else {
  // !!! PONTO CRÍTICO DE TESTE!!!
  // Esta chamada de API ('POST /openfield/v1/webhooks') não está publicamente documentada.
  // Se esta chamada falhar, o fluxo Híbrido é inválido e você deve reverter para o fluxo Manual.
  try {

```

```

    const createWebhookResponse = await
fetch('https://connect.catapultsports.com/api/v1/webhooks', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${api_token}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    eventType: 'session.completed', // ou 'Activity' 'Updated' [3]
    callbackUrl: callbackUrl
  })
});

if (!createWebhookResponse.ok) throw new Error('Falha ao criar webhook na API da Catapult');

const newWebhook = await createWebhookResponse.json();
webhook_id = newWebhook.webhook_id;
webhook_secret = newWebhook.webhook_secret;

} catch (e) {
  console.error(e);
  return new Response(JSON.stringify({ error: `Falha ao criar webhook: ${e.message}` }), { status:
500 });
}
}

// 7. Salvar o Webhook Secret no Vault
const { data: secretVaultId, error: vaultErr2 } = await supabaseAdmin.rpc('create_team_secret', {
  secret_value: webhook_secret,
  secret_name: `catapult_webhook_secret_club_${club_id}`
});
if (vaultErr2) throw vaultErr2;

// 8. Atualizar o registro final
await supabaseAdmin
  .from('club_integrations')
  .update({
    webhook_secret_vault_id: secretVaultId,
    catapult_webhook_id: webhook_id
  })
  .eq('webhook_uuid', integration.webhook_uuid);

return new Response(JSON.stringify({ status: 'connected' }), { status: 200 });
};

```

Endpoint de conexão concluído. 90% feito (lógica real), 10% (chamada de API) precisa de teste real.

🧱 ②, ④, ⑧: Endpoint de Webhook (/api/catapult/webhook/:club_id)

Esta é a função pública que recebe os dados da Catapult.²¹

Arquivo: `supabase/functions/catapult-webhook/index.ts`

TypeScript

```
import { createClient } from 'npm:@supabase/supabase-js@2'
import { Hono } from 'jsr:@hono/hono'
import { MOCK_CATAPULT_PAYLOAD, normalizeMetrics } from '../_shared/normalization_service.ts'

const app = new Hono()
const MOCK_MODE = Deno.env.get('MOCK_MODE') === 'true';

// Roteamento dinâmico para capturar o UUID
app.post('/catapult-webhook/:webhook_uuid', async (c) => {
  const { webhook_uuid } = c.req.param()

  const supabaseAdmin = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
  );

  let club_id: string;
  let webhook_secret: string;
  let integration_id: string;

  try {
    // 1. Identificar o Clube pelo UUID
    const { data: integration, error: intErr } = await supabaseAdmin
      .from('club_integrations')
      .select('id, club_id, webhook_secret_vault_id')
      .eq('webhook_uuid', webhook_uuid)
      .single();

    if (intErr) throw new Error('Integração não encontrada');
    club_id = integration.club_id;
    integration_id = integration.id;

    // 2. Buscar o Segredo do Webhook no Vault
    const { data: secret, error: secretErr } = await supabaseAdmin.rpc(
```

```

    'get_secret_by_id', { secret_id_in: integration.webhook_secret_vault_id }
  );
  if (secretErr) throw new Error('Erro de configuração de segurança');
  webhook_secret = secret;

  // 3. Validar a Assinatura HMAC [3]
  const signature = c.req.header('X-Catapult-Signature') // [3]
  const rawBody = await c.req.text() // [21, 22, 29]

  // (Implementar lógica de verificação HMAC SHA256 aqui) [30, 31, 32, 33, 34]
  // const isValid = await verifyHMAC(rawBody, signature, webhook_secret)
  // if (!isValid &&!MOCK_MODE) {
  //   await logEvent(supabaseAdmin, integration_id, club_id, false, 401, rawBody);
  //   return c.text('Assinatura inválida', 401);
  // }

  // 4. Processar o Payload (Real ou Mock)
  const payload = MOCK_MODE? MOCK_CATAPULT_PAYLOAD : JSON.parse(rawBody);

  // 5. Mapear o Atleta (De-Para)
  const { data: athlete, error: athleteErr } = await supabaseAdmin
    .from('athletes')
    .select('id')
    .eq('external_catapult_id', payload.athleteId)
    .eq('club_id', club_id) // Segurança multi-tenant
    .single();

  if (athleteErr) throw new Error(`Atleta não mapeado: ${payload.athleteId}`);

  // 6. Inserir a Sessão (Upsert previne duplicatas)
  const { data: session, error: sessionErr } = await supabaseAdmin
    .from('sessions')
    .upsert({
      club_id: club_id,
      athlete_id: athlete.id,
      external_session_id: payload.sessionId,
      // started_at, ended_at (virão do payload)
      raw_json: payload
    }, { onConflict: 'club_id, external_session_id, athlete_id' })
    .select('id')
    .single();

  if (sessionErr) throw sessionErr;

  // 7. Normalizar e Inserir Métricas
  const normalized = normalizeMetrics(payload.metrics);

```

```

const metricRecords = normalized.map(m => ({
  session_id: session.id,
  athlete_id: athlete.id,
  timestamp: m.timestamp,
  metric: m.metric,
  value: m.value,
  unit: m.unit,
  // (Você pode adicionar 'zone' como uma coluna JSONB ou 'text' na tabela 'metrics')
}));

const { error: metricsErr } = await supabaseAdmin
  .from('metrics')
  .insert(metricRecords);

if (metricsErr) throw metricsErr;

// 8. Logar sucesso e retornar 200
await logEvent(supabaseAdmin, integration_id, club_id, true, 200, rawBody);
return c.json({ received: true });

} catch (err) {
  console.error(`Erro no Webhook ${webhook_uuid}:`, err.message);
  // @ts-ignore
  await logEvent(supabaseAdmin, integration_id, club_id, false, 500, err.message);
  return c.text(err.message, 500);
}
})

// Função de log (Sua etapa ③)
async function logEvent(client: any, int_id: string, club_id: string, valid: boolean, status: number,
body: string) {
  await client.from('webhook_logs').insert({
    integration_id: int_id,
    club_id: club_id,
    signature_valid: valid,
    status_code: status,
    request_body: body
  });
}

Deno.serve(app.fetch)

```

Endpoint de recebimento concluído. 100% feito, pode ser totalmente testado com `MOCK_MODE=true`.

